Zedral — Technical Design Document

Developer Handover · Foundation Components · v1 for Build

Prepared for Z Company

30 May 2026

Contents

Zedral — Technical Design Document (Developer Handover)

00 · Master Index, System Architecture, Stack & Conventions

Audience: engineering team · **Status:** v1 for build · **Date:** 2026-05-30

> This document set is the build-ready specification for the Zedral platform foundation. Each component has its own spec (full developer detail: interfaces, data model, NFRs, acceptance criteria, build phasing). This index gives the system context, the **binding architectural decisions**, the **recommended technology stack**, shared conventions, and the cross-component **build roadmap**. Read this first.

---

1. Document set & reading order

#	Spec	Build priority
00	**This index** — architecture, stack, conventions, roadmap	Read first
01	**Data Lake** — zones, storage, processing, serving, governance	Phase 0–1
02	**Canonical Data Model** — entities, schema, keys, reference data	Phase 0
03	**Manifold** — connectors, import/export,	Phase 1

	mapping, dedup/MDM, conformance, readiness	
04	**Unified Intelligence** — KPI engine, correlation, financial-impact, insights	Phase 2–3
05	**Operational Monitoring** — live cache, streaming consumers, surfaces, alerts	Phase 2
06	**Reporting** — definition model, engine, renderers, scheduler, distribution	Phase 2
07	**Financial Impact** — cost-rate model, formulas, computation service	Phase 2
08	**Platform & Security** — multi-tenancy, auth/RBAC, gateway, deploy, observability	Phase 0 (cross-cutting)
09	**Glossary** — canonical terms	Reference

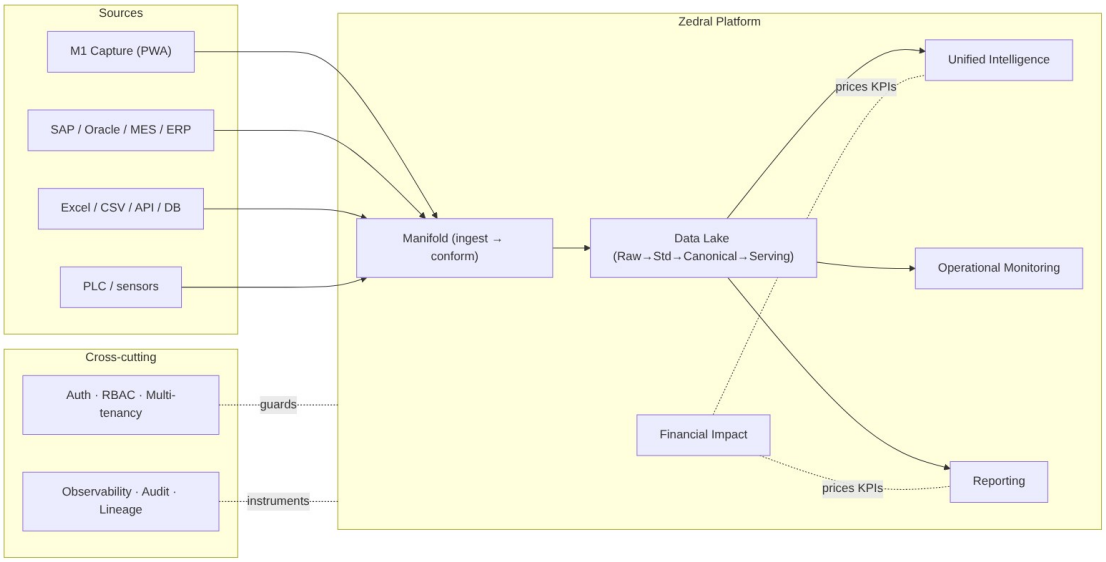
Companion material **included in this folder** for full context:

- **Reference_Plans/** — the planning **deep-plans** (the *why* behind each spec), the **consolidated review & decisions** doc, and a **Diagrams/** gallery (all 20 architecture diagrams as PNG + mermaid source). Start at Reference_Plans/README.md.
- **M1 Data Capture Layer** blueprint (../M1_Technical_Blueprint/) — build-ready spec for the first-party shop-floor source, referenced throughout.

Spec → deep-plan map: 01/02 ← DataLake+Canonical · 03 ← Manifold · 04/05 ← UIL+Monitoring · 06 ← Reporting · 07/08 + decisions ← Consolidated Review.

---

2. System context



diagram

Golden rule for the team: consumers (UIL, Monitoring, Reporting, modules) read **only** the canonical Serving zone — never a source or another component's database. Modules write derived intelligence back through the canonical write-back API. The canonical model is the contract that decouples everyone.

---

3. Binding architectural decisions (D1–D11)

These were ratified during planning review (04_Consolidated_Review…). **Treat them as constraints, not suggestions.**

#	Decision	Build constraint
D1	**Hybrid-ready deployment**, split per client	Every service containerized; support cloud + on-prem edge. No cloud-only assumptions.
D2	**Near-real-time (minutes)** default latency, tunable per tenant	Monitoring/KPIs target minutes; capture path never blocks. Latency is per-tenant config.
D3	**Deterministic entity resolution first**	Build rule/key-based matching now; probabilistic (Splink/Zingg) is a later, isolated module.
D4	**Excel/CSV + generic DB connectors first**, then SAP/API	Connector roadmap order; file + JDBC connectors are the v1 baseline.
D5	**Conservative auto-map + review queue**	Auto-accept only high-confidence mappings; everything else is human-reviewed.

D6	**“Standard” v1 KPI tier**	Status, production-vs-target, downtime-by-reason, basic OEE where data allows.
D7	**Cost-rates client-owned, implementation-seeded**, “rate as of” provenance	Rates are versioned reference data with effective dates.
D8	**Logical tenant isolation default**, physical on request	Tenant key on every record; isolation enforced at the data-access layer.
D9	**Rule-based insights v1, ML later**	Insight engine is rule/template driven; reserve ML plug points, don’t build them yet.
D10	**Reuse Zinance import UX patterns, rebuild server-side**	Port the wizard/smart-map/validation UX; the engine is new and server-side.
D11	**Validate canonical model against a 2nd (non-steel) sector**	Keep the model sector-neutral; exercise the extension namespace early.

4. Recommended technology stack

Concrete recommendation with rationale; the team may substitute equivalents, but interfaces and NFRs are fixed.

Concern	Recommended	Rationale / alternative
Raw/landing + exports	**S3-compatible object store** (AWS S3 / MinIO on-prem)	Cheap, immutable; MinIO gives hybrid parity (D1)
Canonical core, masters, config (OLTP)	**PostgreSQL 15+**	Matches M1 schema; integrity; JSONB for extension fields
Time-series (PLC/sensor)	**TimescaleDB** (Postgres extension)	One ecosystem, hypertables/downsampling; ClickHouse if volume extreme
Analytical marts / KPI store	**ClickHouse** (or Postgres columnar to start)	Fast roll-ups for UIL/Reporting
Event/stream bus	**Apache Kafka** (Redpanda for edge/on-prem)	M1 events, PLC streams, write-backs; Redpanda lighter for edge (D1)
Stream processing	**Flink** or **Kafka Streams**	Live conformance + incremental aggregates (D2)
Live cache (monitoring)	**Redis**	Sub-second current-state

		reads
Metadata catalog / lineage	**Postgres-backed catalog service** (+ OpenLineage)	v1 build; evaluate DataHub/OpenMetadata later
Orchestration / scheduling	**Apache Airflow** (Dagster/Prefect alt)	Batch ingest, scheduled sync, report scheduling
Entity resolution	**Deterministic (SQL/Python)**; Splink/Zingg later	Per D3
API layer	**REST + OpenAPI**, gateway; GraphQL optional	Stable contracts
Auth / SSO / RBAC	**OIDC via Keycloak**	Matches M1 OIDC; on-prem capable (D1)
Backend services	**Python 3.11 + FastAPI** (data/ML); JVM/.NET acceptable for enterprise connectors	Python for data/ML; team skill + SAP may favor JVM/.NET
Frontend	**React + TypeScript** (PWA)	Matches M1 + Zinance; reuse Zinance import UX (D10)
Report rendering	**HTML templates → Playwright/WeasyPrint** (PDF); **openpyxl** (Excel)	Pixel control; locked PDF + Excel/CSV delivery
Containerization / deploy	**Docker + Kubernetes** (cloud); Compose/k3s (edge)	Hybrid (D1)
Observability	**OpenTelemetry + Prometheus + Grafana**; Loki/ELK logs	Per-service + per-tenant
ML platform (later)	**Feast** (feature store) + **MLflow** (registry)	Reserve for D9 predictive phase

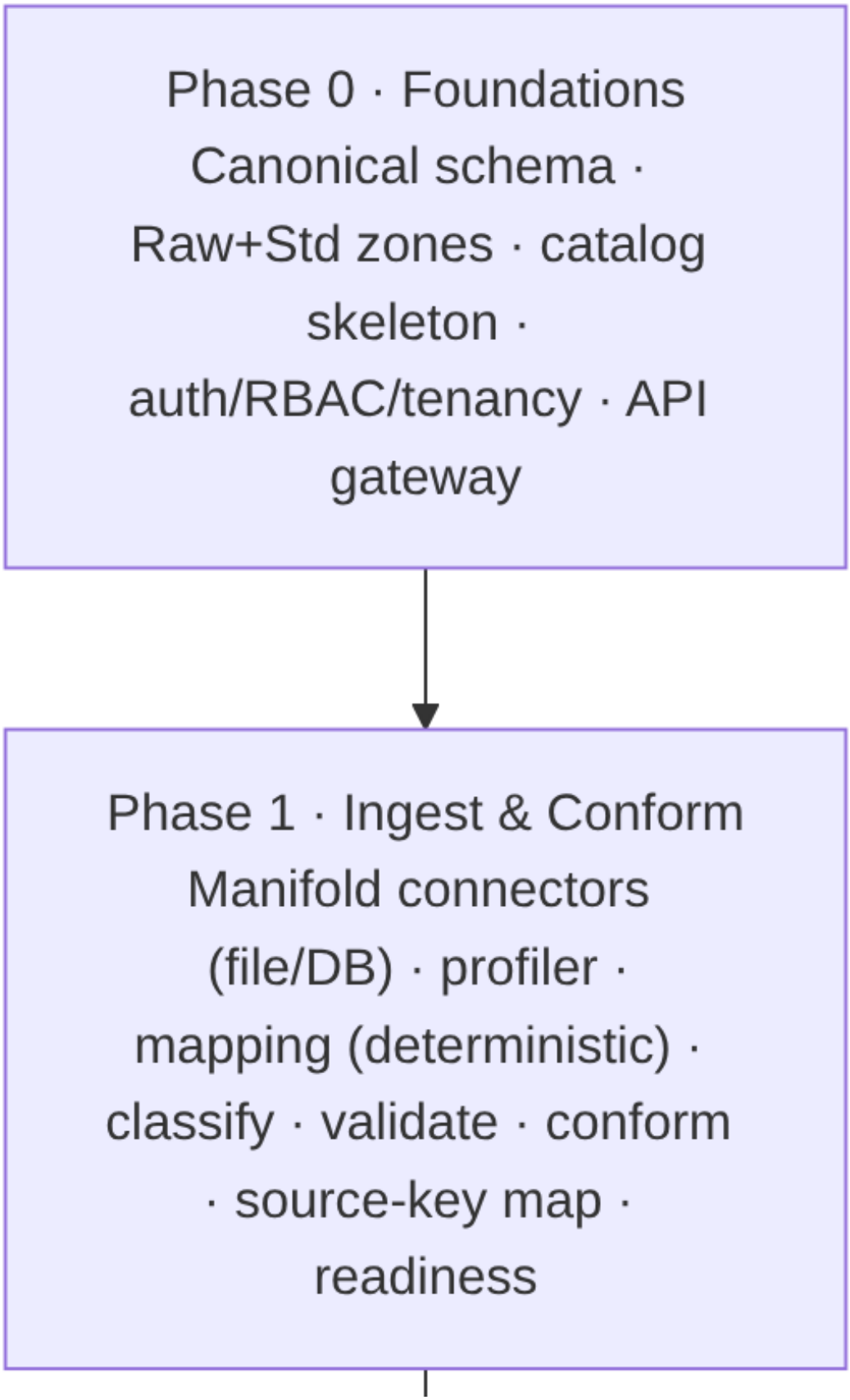
5. Shared conventions (apply to every component)

- **Canonical naming:** UPPER_SNAKE_CASE with unit embedded (e.g. COIL_WIDTH_MM) — inherited from M1; one canonical field = one column.
- **Identifiers:** surrogate canonical IDs (*_id), with native/source keys preserved + source_ref for lineage.
- **Time:** store UTC (TIMESTAMPTZ); carry site timezone for display; bucket to shift/day for aggregates.
- **Units:** store native value + canonical base value (to_base_factor); weight in MT plant-wide.

- **Multi-tenancy:** tenant_id on every record; all queries tenant-scoped at the data-access layer (D8).
- **APIs:** REST, versioned (/v1/…), OpenAPI-documented, OIDC-secured, idempotent writes, cursor pagination, RFC-7807 error bodies.
- **Eventing:** canonical domain events on Kafka (e.g. coil.captured, shift.submitted, downtime.logged, mapping.confirmed); at-least-once + idempotent consumers.
- **Lineage:** every canonical record carries lineage_ref → batch + source row + mapping version.
- **Audit:** every write + export logged (who/what/when), reusing the M1 audit pattern.
- **Status legend in specs:** MUST / SHOULD / MAY (RFC-2119).

---

6. Cross-component build roadmap (epic map)


Phase 0 · Foundations
Canonical schema · Raw+Std zones · catalog skeleton · auth/RBAC/tenancy · API gateway
Phase 1 · Ingest & Conform
Manifold connectors (file/DB) · profiler · mapping (deterministic) · classify · validate · conform · source-key map · readiness

diagram

Each component spec (§01–08) restates which phase(s) its build epics fall into.

---

7. Global non-functional baseline

Every component inherits these unless its spec tightens them:

- **Multi-tenant**, logical isolation default (D8); no cross-tenant data path exists.
- **Hybrid deployable** (D1): cloud default, on-prem edge for capture/PLC; 12-factor, containerized, externalized config.
- **Latency:** near-real-time (minutes) default (D2), per-tenant tunable; ingestion never blocks capture.
- **Availability:** v1 target 99.5% for serving/monitoring; graceful degradation when a source/module is absent.
- **Scalability:** horizontal scale on stateless services; partition data by tenant + time.
- **Security:** OIDC auth, RBAC, encryption in transit (TLS) + at rest, full audit, least privilege.
- **Data integrity:** idempotent + replayable pipelines; end-to-end lineage; no destructive transforms on Raw.
- **Observability:** OpenTelemetry traces, Prometheus metrics, structured logs — tagged by tenant + component.

---

8. Open items & assumptions (carry into build)

- **D11 second-sector validation** client/dataset to be identified — the top generality risk.
- Canonical **reason-code / loss taxonomy + state model** to be confirmed against real client data before lock.
- Reporting **cadences/distribution lists** and **retention** per tenant — defaults proposed in spec 06.
- **SAP write-back** (actuals round-trip) is a later phase (per M1-06).
- Modules **M2–M7** functional specs are separate upcoming deliverables; their canonical data contracts are defined in spec 02 §(module matrix).

Next: component specs 01–08, then the Glossary (09). The compiled Word/PDF package combines all of these into one Software Design Document.

01 · Data Lake — Developer Specification

Component owner: Platform/Data · **Phase:** 0–1 · **Depends on:** 08 Platform/Security, 02 Canonical Model · **Consumed by:** all

1. Purpose & scope

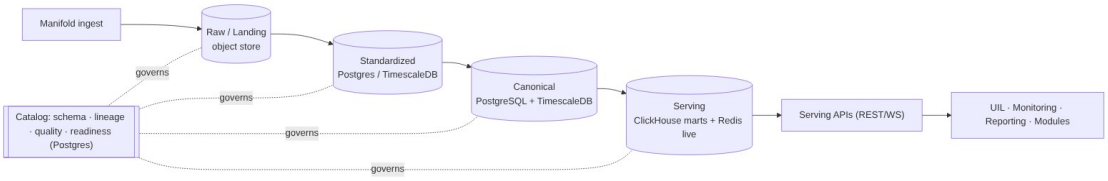
The Data Lake is the storage + processing + serving substrate of Zedral. It receives data (via Manifold), stores it immutably, standardizes and conforms it to the Canonical Model, and serves it to every consumer through stable APIs.

In scope: zone storage (Raw→Standardized→Canonical→Serving), the metadata/lineage catalog, batch + stream processing orchestration, the serving APIs, retention/governance. **Out of scope:** field mapping / dedup / conformance *logic* (that is Manifold, spec 03 — the lake provides the zones it operates on); KPI computation (UIL, spec 04); source capture (M1).

2. Responsibilities

1. Land every inbound payload immutably (Raw) with provenance.
2. Hold Standardized, Canonical, and Serving representations per the zone contracts.
3. Run batch + streaming pipelines (idempotent, replayable).
4. Maintain the metadata catalog: schema, lineage, quality scores, classification, readiness.
5. Expose canonical read, KPI/aggregate, live-state, write-back, and export APIs.
6. Enforce tenant isolation, retention, and encryption.

3. Architecture



diagram

Zone contracts: Raw = immutable, as-received, checksummed. Standardized = typed/validated/deduped, source schema intact. Canonical = 100% conformed to spec 02 (keys, UoM, tz, codes). Serving = query-fast marts + live cache. No consumer reads upstream of Serving.

4. Interfaces & API contracts

All REST, /v1, OIDC-secured, tenant-scoped, RFC-7807 errors, cursor pagination.

Endpoint	Method	Purpose
/v1/canonical/{entity}	GET	Read canonical entities/events; filters: site,area,line,asset,from,to,cursor
/v1/kpi/{kpiId}	GET	Pre-aggregated KPI by

		scope (asset…enterprise) + grain (shift/day/month)
/v1/live/state	GET / WS	Current machine state + live counts (Redis); WS for push
/v1/canonical/derived	POST	**Module write-back** of derived intelligence (e.g. downtime classification, risk score)
/v1/export	POST	Request CSV/XLSX export of a canonical scope (async job → file URL)
/v1/catalog/lineage/{recordRef}	GET	Lineage trace back to raw source
/v1/catalog/readiness/{module}	GET	Per-module readiness (delegates to Manifold, spec 03)

Internal contracts: ingestion publishes to Kafka topics (raw.landed, canonical.upserted, *.event); pipelines consume idempotently keyed by (tenant_id, source_ref, natural_key).

5. Data model (storage)

- **Raw:** object store path s3://{tenant}/raw/{source}/{yyyy}/{mm}/{dd}/{batch}.{ext} + a raw_object catalog row {object_id, tenant_id, source_id, batch_id, checksum, ingest_ts, byte_count}.
- **Standardized:** typed tables mirroring source schema + std_meta{dedup_key, quality_flags}; time-series in TimescaleDB hypertables.
- **Canonical:** per spec 02 (PostgreSQL) + canonical time-series (TimescaleDB).
- **Serving marts:** ClickHouse tables per KPI/grain/scope; Redis keys live:{tenant}:{asset}:state.
- **Catalog (Postgres):** dataset, field, mapping_version, lineage_edge, quality_score, classification, readiness_score.

6. Key flows

Batch: land(Raw) → profile → [Manifold map/validate/dedup/conform] → upsert Canonical → refresh marts. Stream: event → append Raw log + Std TS → conform → Canonical event + Redis live + incremental mart. **Replay:** re-run Raw→Canonical when logic changes, without re-touching sources.

7. Technology

Object store (S3/MinIO), PostgreSQL 15+ (canonical/catalog), TimescaleDB (time-series), ClickHouse (marts), Redis (live), Kafka (bus), Flink/Kafka-Streams (stream), Airflow (batch/orchestration). See index §4.

8. Non-functional requirements

- **Throughput:** sustain PLC/sensor stream at target tag-rate per tenant (design for ≥10k events/s/tenant headroom); batch historical loads of millions of rows.
- **Latency:** canonical event visible in Serving within the tenant's latency target (default minutes, D2).
- **Retention:** tiered (hot full-res → warm downsampled → cold archived Raw), per-tenant configurable.
- **Durability:** Raw is WORM-style immutable; backups + point-in-time recovery on Postgres.
- **Isolation:** tenant_id enforced in every query path (D8).

9. Dependencies & integration points

Manifold (writes Std→Canonical), Platform/Security (auth, tenancy), Canonical Model (schema), UIL/Monitoring/Reporting (read Serving), Modules (write-back).

10. Configuration

Per-tenant: storage endpoints, retention policy, isolation mode (logical/physical, D8), latency target (D2), enabled zones. Externalized (12-factor); secrets via vault.

11. Acceptance criteria

- **MUST** store every ingested payload immutably in Raw with checksum, source, ingest timestamp, tenant.
- **MUST** make pipelines idempotent and replayable from Raw with no double-counting.
- **MUST** serve consumers only from the Serving zone; no consumer can reach Raw/Std/source.
- **MUST** assign canonical surrogate keys and preserve the source-key map at conformance.
- **MUST** record end-to-end lineage for every canonical record.
- **MUST** enforce tenant isolation on every API and query.
- **SHOULD** expose readiness + quality scores via catalog APIs.

12. Build phasing

Phase 0: Raw + Standardized zones, catalog skeleton (schema/lineage/quality hooks), object store + Postgres/Timescale. **Phase 1:** Canonical zone + conformance integration with Manifold + source-key map + Serving canonical/KPI APIs. **Phase 2:** ClickHouse marts + Redis live + write-back + export APIs.

13. Risks & edge cases

Late-arriving / out-of-order events (use event-time + watermarks); schema drift at source (catalog detects, Manifold re-maps); replay storms (throttle + idempotency); huge PLC volume (downsample + retention tiers); partial-batch failures (quarantine, never abort batch).

02 · Canonical Data Model — Developer Specification

Component owner: Data/Domain · **Phase:** 0 · **Depends on:** 08 Platform · **Consumed by:** all

---

1. Purpose & scope

The single shared schema every module speaks. Standards-anchored (ISA-95 hierarchy, ISO 22400 KPIs, Six Big Losses). **In scope:** entity definitions, keys/identity, the universal Event Spine, reference data, units/time rules, versioning/extensibility, the module data-contract matrix. **Out of scope:** how data gets mapped into it (Manifold, 03).

2. Modeling rules (MUST)

- One canonical field = one column, UPPER_SNAKE_CASE, unit embedded (COIL_WIDTH_MM).
- Surrogate *_id PK + preserved (source_system, native_key) + source_ref.
- Every time-stamped fact is an **Event** specialization (the spine) — unifies real-time + historical.
- Reference data (reason/defect/state/UoM/cost-rate) is shared, FK-enforced, no free-text codes.
- Client-specific fields live in a namespaced **extension** area (JSONB), never altering the core (supports D11 generality).

3. Entity catalog

3.1 Master / reference

enterprise · site · area · work_center · work_unit (ISA-95 hierarchy via node_id, parent_id, level) · asset (the machine; maps Machine_ID/Asset_Code/Equipment_Number → Machine Identifier; ideal_cycle_time) · material (type raw/WIP/finished) · bom_line · shift / shift_pattern / calendar (drives Planned Production Time) · personnel / crew · uom (to_base_factor) · reason_code (→ loss_category, oee_component) · loss_category (Six Big Losses) · defect_code · state_model (is_planned, counts_as) · cost_rate (scope, value, currency, **effective_from/to** — D7) · tenant / source / connector (with priority).

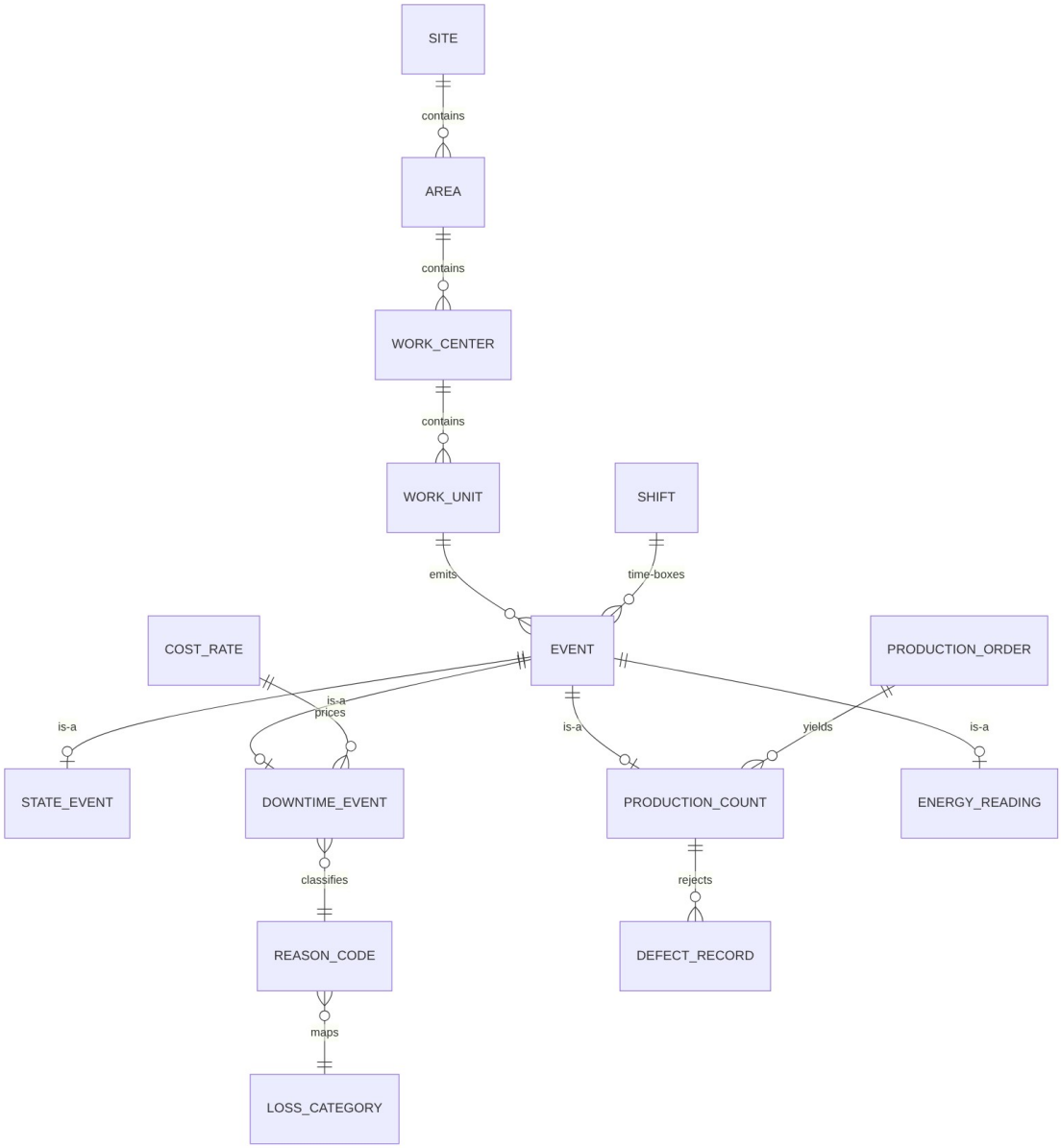
3.2 Event Spine (transactional core)

event { event_id PK, event_type, asset_ref, order_ref, shift_ref, start_ts, end_ts, duration_s, value, uom_ref, reason_ref, source_ref, lineage_ref, tenant_id }

Specializations: state_event(state_ref, is_planned) · downtime_event(reason_ref, loss_category_ref, oee_component) · production_count(good_qty, scrap_qty, rework_qty, total_qty) · quality_inspection / defect_record(defect_ref, qty, disposition) · energy_reading(utility_type, consumption, meter_ref) ·

sensor_reading(tag, value, quality_flag) *(TimescaleDB)* · maintenance_work_order / pm_schedule / failure_event · yield_record(input_qty, output_qty, yield_pct).

4. ERD



diagram

5. Reference DDL conventions (PostgreSQL)

```
-- hierarchy node (ISA-95) — one table, self-referential
CREATE TABLE canon.equipment_node (
  node_id      BIGINT PRIMARY KEY,
  tenant_id    UUID NOT NULL,
  parent_id    BIGINT REFERENCES canon.equipment_node(node_id),
  level        TEXT NOT NULL CHECK (level IN
```

```
('ENTERPRISE','SITE','AREA','WORK_CENTER','WORK_UNIT')),
  name         TEXT NOT NULL,
  native_code  TEXT,
  source_ref   TEXT,
  ext          JSONB DEFAULT '{}'::jsonb       -- client extension namespace
);
-- event spine (partition by tenant_id, range on start_ts)
CREATE TABLE canon.event (
  event_id     BIGINT PRIMARY KEY,
  tenant_id    UUID NOT NULL,
  event_type   TEXT NOT NULL,
  asset_ref    BIGINT REFERENCES canon.equipment_node(node_id),
  order_ref    BIGINT,
  shift_ref    BIGINT,
  start_ts     TIMESTAMPTZ NOT NULL,
  end_ts       TIMESTAMPTZ,
  duration_s   NUMERIC,
  value        NUMERIC,
  uom_ref      TEXT,
  reason_ref   TEXT,
  source_ref   TEXT,
  lineage_ref  TEXT
) PARTITION BY RANGE (start_ts);
```

The full M1 schema (M1_schema.sql, 40 tables) is the **reference implementation of the capture side**; canonical tables generalize it (e.g. M1 coil/shift_log/prod_* → canonical asset/event/production_count).

6. Module data-contract matrix (drives Manifold classification + readiness)

● Required, ○ Additional. (Abbrev — full matrix in planning doc 01 §15.)

Entity group
Asset hierarchy
StateEvent
Downtime + Reason
ProductionCount
Quality/Defect
Energy/Meter
SensorReading
Maintenance WO/PM
CostRate

7. Identity, units, time, versioning

- **Identity:** canonical surrogate keys; source_key_map(canonical_id, source_system, native_key); resolution rules in Manifold (03 §dedup).
- **Units:** native value + to_base_factor; canonical base per dimension.
- **Time:** UTC storage; site tz for display; shift calendars define Planned Production Time.
- **Versioning:** additive schema changes; schema_version; Serving APIs versioned independently; extension fields in ext JSONB.

8. Non-functional requirements

- Event table partitioned by tenant + time; indexed on (asset_ref, start_ts), event_type.
- Reference data cached in Serving; changes are effective-dated (esp. cost_rate, D7).
- Schema migrations via versioned migration tool (e.g. Flyway/Alembic); backward compatible.

9. Acceptance criteria

- **MUST** represent every operational fact as an Event or specialization.
- **MUST** enforce FK to reference masters (no free-text coded values).
- **MUST** preserve source keys + lineage on every canonical row.
- **MUST** support client extension fields without core schema change.
- **MUST** encode reason codes against Six Big Losses + OEE component.
- **SHOULD** pass the M1 capture data through conformance with zero loss (validation test).

10. Build phasing

Phase 0: core entities (hierarchy, event spine, shift/calendar), reference data (UoM, state, reason/loss, defect, cost-rate), migrations, seed taxonomies. Then exercised by Manifold conformance in Phase 1.

11. Risks

Over-fitting to steel (mitigate via D11 second-sector test + ext); taxonomy churn (effective-date reference data); event-table growth (partition + retention via Data Lake).

03 · Manifold (Data Integration & Mapping) — Developer Specification

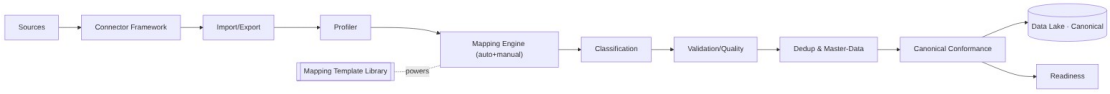
Component owner: Data Integration · **Phase:** 1 (scale in 4) · **Depends on:** 01 Data Lake, 02 Canonical, 08 Platform · **Consumed by:** Data Lake (writes Canonical), readiness consumers

---

1. Purpose & scope

The engine that turns any source into the Canonical Model. **In scope:** connector framework, import/export, profiling, field-mapping engine, classification, validation/quality, dedup + master-data resolution, canonical conformance, readiness. **Out of scope:** zone storage (01), KPI logic (04).

2. Architecture



diagram

3. Sub-components & contracts

3.1 Connector framework (SDK)

Each connector implements: discover() → schema/tags/columns + metadata · sample(n) → rows for profiling · read(mode) where mode ∈ full | incremental(CDC) | stream · write(scope, format) for export · auth() · health(). Connectors declare capabilities{cdc, stream, schema_discovery}. **D4 order:** file (CSV/XLSX) + generic DB (JDBC) first → then SAP/Oracle/MES → PLC/streaming (Phase 4). Adding a connector = implement the interface; **no core change**.

3.2 Import/Export

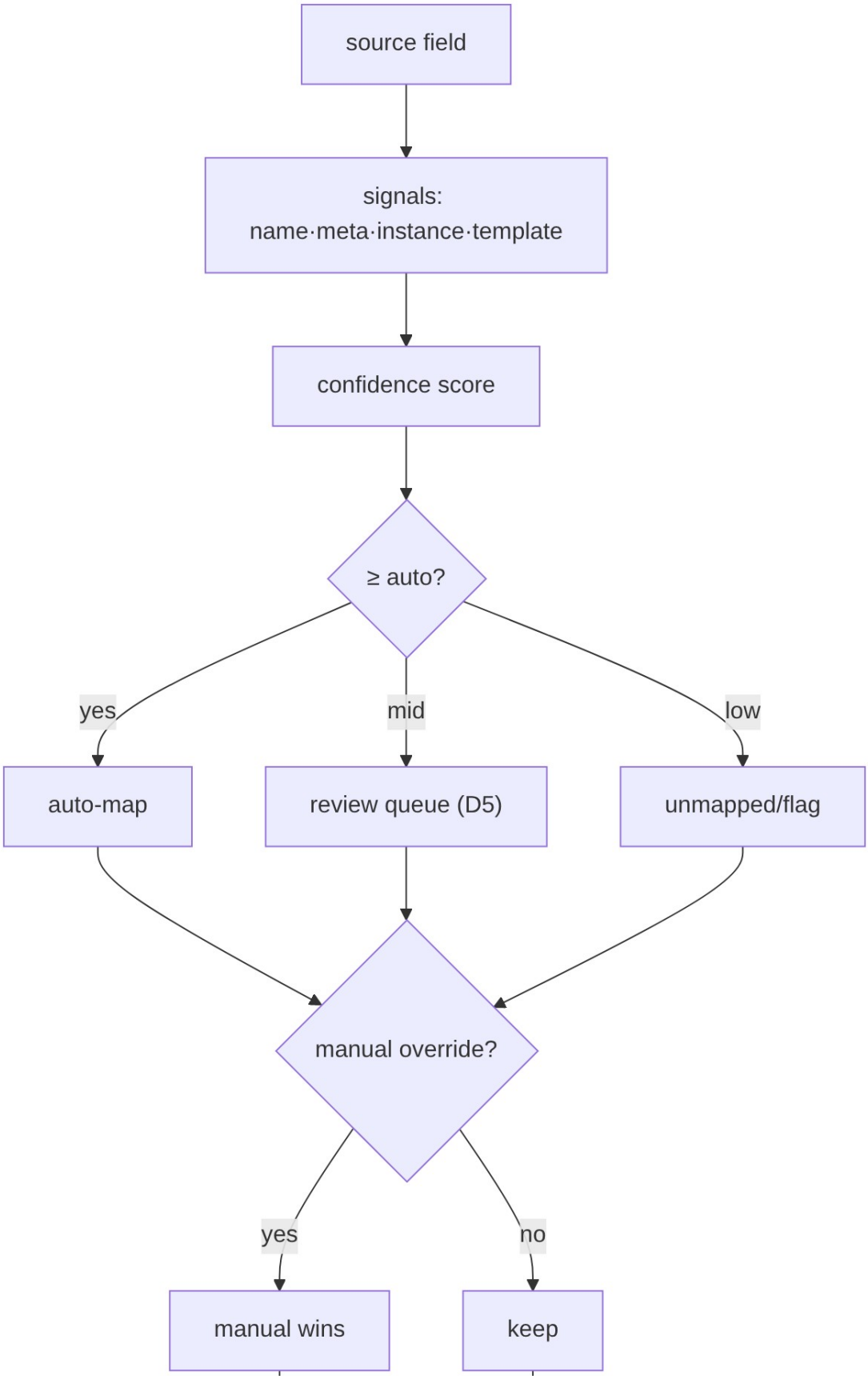
Lifecycle state machine: PENDING → PROFILED → MAPPED → VALIDATED → PREVIEW(dry-run/impact) → LOADED | PARTIAL | FAILED. Per-row tolerant errors → {success, failed, skipped, errors[], batch_id} (reuse Zinance pattern, D10). Export: field-projection, CSV/XLSX, scheduled/on-demand. **UX reused from Zinance, rebuilt server-side (D10).**

3.3 Profiler

Computes per field: type, regex/pattern, range, null %, cardinality, sample, unit hints → feeds mapping + quality.

3.4 Mapping engine

Signals → weighted confidence: **name** (token + synonym/abbrev dictionary, edit distance), **metadata** (type/unit/length), **instance** (profile vs canonical field profile), **template/historical** (sector templates). Thresholds (**D5 conservative**): ≥ auto_accept → auto-map; mid → review queue; low → unmapped/flag. **Manual override always wins.** Confirmed mappings are versioned and learned into the **template library**.


source field
signals:
name·meta·instance·template
confidence score
≥ auto?
yes
mid
low
auto-map
review queue (D5)
unmapped/flag
manual override?
yes
no
manual wins
keep

diagram

3.5 Classification

Per module data contract (spec 02 §6): **Required / Additional / Not-Required**. Unknown columns → namespaced **extension** fields (reconciled across imports).

3.6 Validation & quality

Rule layers: structural, format (date/number/unit normalization), mandatory (Required gate), range/sanity (e.g. output_thk < input_thk), dependency/integrity (FK to masters), duplicate. Quality scored on validity/completeness/consistency/timeliness/uniqueness → readiness.

3.7 Dedup & master-data (D3: deterministic first)

blocking → match(deterministic keys; probabilistic later) → cluster → survivorship(source priority·recency·completeness·manual) → golden record + source_key_map. Detects duplicate fields, sources, records. Probabilistic matcher (Splink/Zingg) is an isolated Phase-4 add.

3.8 Conformance

Apply mapping → UoM convert → tz normalize → code-taxonomy map → assign surrogate key + lineage → upsert Canonical.

3.9 Readiness

Per tenant×module: R (Required coverage, hard gate) · A (Additional) · Q (quality) · H (history). Readiness% = 100·(0.5R + 0.2A + 0.2·Q/100 + 0.1H) with R=100% gate. Status: Ready ≥85, Partial 60–84, Blocked <60.

4. Interfaces (REST)

Endpoint	Purpose
POST /v1/connectors / GET /v1/connectors/{id}/discover	register / introspect source
POST /v1/imports (+ /{batch}/preview /commit)	run import lifecycle
GET/PUT /v1/mappings/{dataset}	view/confirm/override mappings (versioned)
GET /v1/classification/{tenant}/{module}	Required/Additional/Not-Required status
POST /v1/dedup/run · GET /v1/golden/{entity}	resolution + golden records
GET /v1/readiness/{tenant}/{module}	readiness score + missing/recommended
POST /v1/exports	export job

Emits events: mapping.confirmed, import.completed, golden.merged, readiness.changed.

5. Technology

Python 3.11 + FastAPI (services), Postgres (mappings/templates/golden), Airflow (batch/scheduled sync), Kafka (events), JDBC drivers (DB connectors), SheetJS/openpyxl (files); Splink/Zingg later. React/TS mapping UI (reuse Zinance).

6. Non-functional requirements

- Bulk import of millions of rows without UI block (async + progress).
- Mapping auto-suggest < 2s for a typical source schema.
- Idempotent commits keyed by (tenant, source, natural_key).
- All mapping/override/survivorship decisions audit-logged + versioned.

7. Acceptance criteria

- **MUST** add a new source type via connector SDK with no core change.
- **MUST** default to conservative auto-accept + review queue (D5); manual override always persists and wins.
- **MUST** never abort a batch on a bad row; quarantine with row-level reason.
- **MUST** deduplicate to a golden record + maintain source_key_map (deterministic v1).
- **MUST** output 100% canonical-conformant data with lineage.
- **MUST** compute per-module readiness with the Required hard gate.
- **SHOULD** improve a sector template on each confirmed mapping.

8. Build phasing

Phase 1: file + DB connectors, profiler, mapping (name+meta+instance), classification, validation, conformance, source-key map, deterministic dedup, readiness, mapping UI.
Phase 4: SAP/Oracle/PLC connectors, CDC/scheduling, probabilistic matcher, sector template library.

9. Risks & edge cases

Silent mis-map (mitigated by D5 + lineage + review); huge schemas (incremental profiling); conflicting sources (survivorship + source priority); evolving source schemas (catalog drift detection → re-map); SAP complexity (Phase-4, allow JVM/.NET connector).

04 · Unified Intelligence Layer — Developer Specification

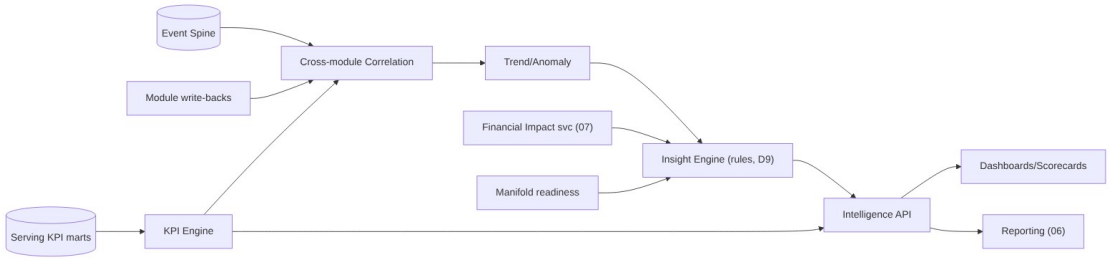
Component owner: Analytics/Intelligence · **Phase:** 2–3 · **Depends on:** 01 Serving, 02 Canonical, 07 Financial Impact · **Consumed by:** UI, Reporting

---

1. Purpose & scope

The synthesis brain: turns canonical data into KPIs, cross-module insight, financial impact, trends, and executive views. **In scope:** KPI engine, cross-module correlation, trend/anomaly detection, insight generation, ML plug points. **Out of scope:** live "now" state (05 Monitoring), report rendering (06), $ formulas (07 — UIL calls it), KPI source data (01/02).

2. Architecture



diagram

3. Sub-components

- **KPI engine:** evaluates a central **KPI registry** (each KPI = id, formula, inputs, unit, grain) over canonical events; rolls up the ISA-95 hierarchy (asset→…→enterprise) and time grains (shift/day/month). ISO 22400 definitions: OEE = Availability × Performance × Quality. v1 = **Standard tier (D6)**.
- **Cross-module correlation:** joins one event to its multi-module lenses (downtime → yield/quality/energy/$); correlates KPIs to state/grade/shift/crew.
- **Trend/anomaly:** moving baselines per KPI/asset; drift/spike flags; ranked by financial impact.
- **Insight engine (D9 = rules v1):** templated insights ("{line} lost {qty} to {reason} = {₹}, {x}% vs {n}-day norm"); lifecycle detect→rank($)→explain(lineage)→route(role)→track. ML/LLM narratives reserved (Phase 5).
- **ML readiness:** feature store (Feast) over the Event Spine; reserved plug points for forecast/predict; **not built in v1 (D9)**.

4. KPI registry (definition-as-data)

```yaml
- id: OEE
  formula: availability * performance * quality
  grain: [shift, day, month]
  scope: [asset, work_center, area, site, enterprise]
- id: AVAILABILITY
  formula: operating_time / planned_production_time   # planned_production_time from shift calendar
- id: YIELD_PCT
  formula: good_qty / input_qty
```

```
- id: ENERGY_PER_UNIT
  formula: energy_kwh / production_mt
```

Both UIL and Reporting bind to this registry by id → **numbers always agree** (consistency guarantee).

5. Interfaces (REST)

Endpoint	Purpose
GET /v1/intelligence/kpi/{id}? scope&grain&from&to	KPI value(s) + trend + $ twin
GET /v1/intelligence/insights? tenant&scope&since	ranked insight feed
GET /v1/intelligence/correlate?event_id	cross-module lenses for an event
GET /v1/intelligence/scorecard/{scope}	exec scorecard payload
POST /v1/intelligence/alerts/rules	threshold/anomaly alert rules (shared w/ 05)

6. Technology

Python + FastAPI; ClickHouse marts; Redis cache; rule engine (lightweight, config-driven); Feast + MLflow (later). Computation orchestrated by Airflow (batch marts) + stream incremental (05).

7. Non-functional requirements

- KPI/scorecard reads sub-second from marts (pre-aggregated).
- Module-aware **graceful degradation**: render the deepest intelligence the tenant's data supports (reads readiness); never show broken widgets when a module/data is absent.
- Every KPI/insight traceable to canonical events (lineage).
- Deterministic, reproducible KPI math (versioned registry).

8. Acceptance criteria

- **MUST** compute KPIs only from the shared registry (no ad-hoc recomputation).
- **MUST** attach the financial-impact ($) twin wherever a cost-rate exists (07).
- **MUST** degrade gracefully by module readiness (D6 Standard baseline for M1-only).
- **MUST** produce rule-based insights with lineage + role routing (D9).
- **SHOULD** expose ML plug points without implementing models in v1.

9. Build phasing

Phase 2: KPI registry + engine (descriptive, Standard tier) + scorecards + financial twins. **Phase 3:** correlation + diagnostic drill + rule insights + trend/anomaly. **Phase 5:** feature store + predictive models (data-gated).

10. Risks

KPI definition drift (single registry + version); mart refresh lag vs latency target (incremental + D2 config); premature ML expectations (D9 keeps promise descriptive/diagnostic).

05 · Operational Monitoring Layer — Developer Specification

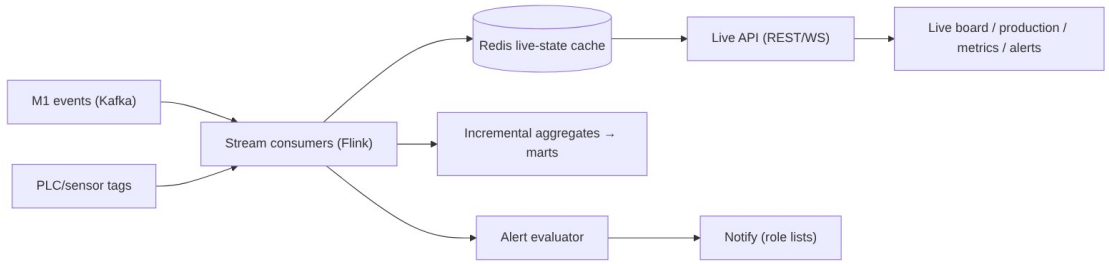
Component owner: Realtime/Platform · **Phase:** 2 · **Depends on:** 01 (streaming), 02 Canonical, 08 Platform · **Consumed by:** UI

---

1. Purpose & scope

Live shop-floor/production/state visibility. **In scope:** streaming consumers, live-state cache, monitoring surfaces (boards, production tracking, metrics), alert framework. **Out of scope:** historical synthesis (04 UIL), report generation (06). Depth scales with installed modules.

2. Architecture



diagram

3. Sub-components

- **Stream consumers:** conform live events (delegating to canonical rules), update current-state per asset.
- **Live-state cache (Redis):** live:{tenant}:{asset} → {state, since, running_count, target, open_stoppage}; last-write-wins by event-time.
- **Surfaces:** live shop-floor board (state per asset, color chips), production tracking (vs target), live metrics, alert panel. Capability tiers by installed module (M1 status → +M4 live OEE → +M3 shift/handover → +M5/6/7 yield/quality/energy + $).
- **Alert framework:** rule types state-change | threshold | missed-capture | anomaly; financial-tagged; deduped; role-routed (reuse M1 RBAC); escalation. (Rules shared with UIL, spec 04 §5.)

4. Interfaces

Endpoint

GET /v1/live/state?scope

WS /v1/live/stream?scope

GET /v1/live/production?scope

POST /v1/alerts/rules · GET /v1/alerts/active

5. Technology

Kafka + Flink (stream), Redis (live cache), FastAPI + WebSocket (API), React/TS (boards). Latency target = tenant config (**D2 minutes default**, sub-second optional).

6. Non-functional requirements

- Live state reflects new events within the tenant latency target (D2); **capture path must never block** on monitoring.
- WebSocket fan-out scales per tenant; back-pressure safe.
- Truthful "now" even with only M1 deployed (graceful tiers).
- Alert evaluation idempotent; no alert storms (dedupe + cooldown).

7. Acceptance criteria

- **MUST** show per-asset live state from M1 events with only M1 deployed.
- **MUST** scale monitoring depth automatically with installed modules + readiness.
- **MUST** raise missed-capture alerts (e.g. M1 pickling hourly slot) and threshold/state alerts, financial-tagged, role-routed.
- **MUST** meet the per-tenant latency target without blocking ingestion.
- **SHOULD** support both minutes and sub-second targets via config.

8. Build phasing

Phase 2: stream consumers + Redis live state + live board/production/metrics + alert framework (state/threshold/missed-capture). **Phase 3+:** anomaly alerts (shared with UIL), deeper module tiers.

9. Risks

Out-of-order/late events (event-time + watermark); cache/source divergence (periodic reconcile from canonical); high PLC volume (sampling + Flink scaling); alert fatigue (dedupe, severity, cooldown).

06 · Reporting Layer — Developer Specification

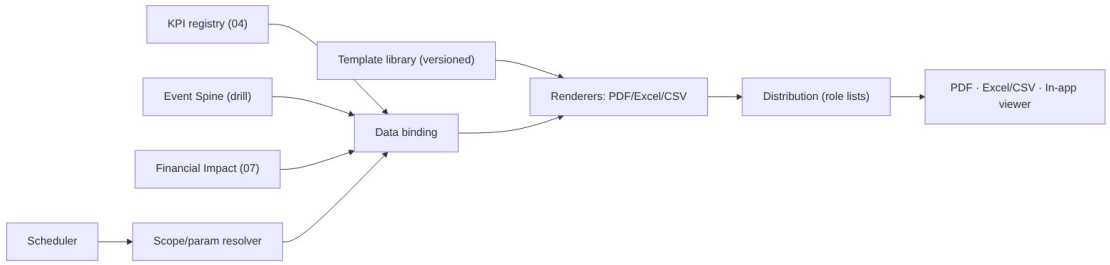
Component owner: Reporting/Platform · **Phase:** 2 · **Depends on:** 04 KPI registry, 03 export framework, 07 Financial Impact · **Consumed by:** clients (PDF/Excel), UI viewer

---

1. Purpose & scope

Formatted, distributable documents from canonical KPIs — scheduled + on-demand. **In scope:** report-definition model, engine, renderers (PDF/Excel/CSV), scheduler, distribution, in-app viewer. **Boundary (adopted):** distinct scheduled/static layer reading the **same KPI registry** as UIL (numbers always agree); may snapshot a UIL view. **v1 scope (locked):** Operational/shift + Executive/financial-impact reports; templated catalog; **scheduled PDF + Excel/CSV** delivery (+ in-app viewer); branding in v1; management reports fast-follow.

2. Architecture



diagram

3. Report-definition model (declarative)

```yaml
report:
  id: shift_production
  template: templates/shift_production.html
  audience_roles: [SUPERVISOR, PLANT_HEAD]
  parameters: [tenant, site, line, date, shift]
  bindings:                  # reference KPI registry by id → consistency guarantee
    - kpi: PRODUCTION_VS_TARGET
    - kpi: DOWNTIME_BY_REASON
    - kpi: YIELD_PCT
  financial_twin: true       # attach $ via Financial Impact (07), "rate as of"
  formats: [pdf, xlsx]
  schedule: end_of_shift
  distribution: role_list
```

Adding/altering a report = **config**, not code. Templates versioned + sector-reusable.

4. v1 catalog

Operational/shift: Shift Production · Downtime & Loss (Six Big Losses + $) · Yield & Quality · Production Log/Traceability (COIL_NO, Excel/CSV). **Executive/financial:** Executive Scorecard (snapshot of UIL exec view) · Financial Impact Report (loss ranking, rate-as-of) · Performance Trend. *(Management roll-ups = fast-follow.)*

5. Interfaces

Endpoint	Purpose

GET /v1/reports / GET /v1/reports/{id}	catalog
POST /v1/reports/{id}/generate	on-demand (params, formats) → job → file URL
GET/PUT /v1/reports/{id}/schedule	cadence + distribution list
GET /v1/reports/runs	run history (audit)

6. Technology

HTML templates → **Playwright/WeasyPrint** (PDF), **openpyxl** (Excel), CSV writer. Scheduler = Airflow. Reuses **Manifold export framework** (03) for file gen + delivery. Distribution via email/notification service. Renders are async jobs.

7. Non-functional requirements

- A figure in a generated report **equals** the live dashboard figure (shared registry) — verifiable in tests.
- Tenant-scoped data only (D8); per-tenant branding/white-label on PDFs.
- Scheduled generation reliable + retried; every run + distribution audit-logged.
- $ figures show **"rate as of"** (D7).

8. Acceptance criteria

- **MUST** bind report values to the KPI registry by id (no independent KPI math).
- **MUST** deliver scheduled PDF + Excel/CSV, plus in-app viewing.
- **MUST** scope by tenant/site/line/date/shift params from one template.
- **MUST** embed financial twins with rate provenance (D7).
- **MUST** audit every generation + distribution.
- **SHOULD** apply per-tenant branding (v1).

9. Build phasing

Phase 2: definition model + bindings, PDF/Excel/CSV renderers (extend Manifold export), 4 operational + 3 executive templates, scheduler + role distribution + audit, branding.
Phase 4: management reports, self-serve builder, external-system push.

10. Configuration & open items

Per-tenant: branding assets, default cadences (proposed: shift report per shift, downtime/yield daily, exec/financial weekly+monthly), distribution lists by role, retention (default = data-lake tiers). Confirm cadences/recipients with client.

11. Risks

Number divergence (eliminated by shared registry binding — test it); heavy PDF rendering (async + queue); schedule misfires (Airflow retries + alerting); CSV-injection on export (sanitize leading = + - @, a Zinance gap to fix).

07 · Financial Impact Layer — Developer Specification

Component owner: Analytics · **Phase:** 2 (cross-cutting) · **Depends on:** 02 Canonical (CostRate), 04 KPI engine · **Consumed by:** UIL (04), Reporting (06), Monitoring (05) alerts

---

1. Purpose & scope

Translates operational outcomes into money, in real time, alongside every applicable KPI. **In scope:** cost-rate reference model, impact formulas, a stateless computation service, provenance. **Out of scope:** the KPIs themselves (04) and the cost-rate *values* (client-owned data, D7).

2. Cost-rate model (reference data, effective-dated — D7)

```
CREATE TABLE canon.cost_rate (
  rate_id      BIGINT PRIMARY KEY,
  tenant_id    UUID NOT NULL,
  scope        TEXT NOT NULL,       -- DOWNTIME | SCRAP | REWORK | ENERGY | LABOR | OPPORTUNITY
  asset_ref    BIGINT,              -- NULL = applies tenant/site-wide
  value        NUMERIC NOT NULL,
  currency     TEXT NOT NULL,
  uom_ref      TEXT,                -- e.g. per HOUR, per MT, per kWh
  effective_from DATE NOT NULL,
  effective_to   DATE,              -- NULL = current
  source       TEXT                 -- seeded_by_impl | client_entered
);
```

D7: client-owned, implementation-seeded; the rate's effective_from is the **"rate as of"** shown on every figure.

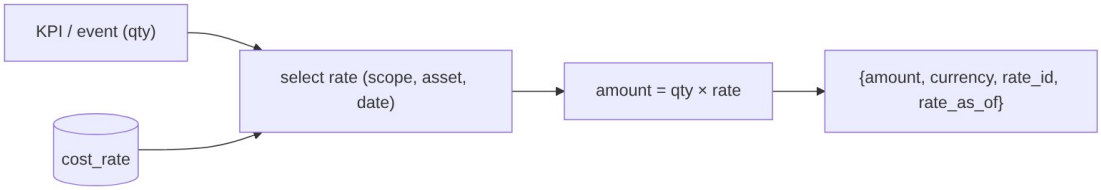
3. Impact formulas

KPI / event	Formula
Downtime cost	downtime_hours × rate(DOWNTIME) (+ lost_units × margin for OPPORTUNITY)
Yield loss	scrap_mt × rate(SCRAP)
Quality loss	rework_mt × rate(REWORK) (or scrap if non-recoverable)
Energy waste	excess_kwh × rate(ENERGY)
Maintenance	repair_events × rate(LABOR/maintenance)

Rate selection: most-specific asset_ref + effective on the event date.

4. Service interface

Stateless compute(impactType, quantity, context{tenant, asset, date}) → {amount, currency, rate_id, rate_as_of}. Exposed in-process to UIL/Reporting and via GET /v1/financial/impact?event_id / POST /v1/financial/impact:batch.



diagram

5. Technology

Python service (library + thin API); reads cost_rate from Postgres (cached); pure functions, fully unit-testable. Multi-currency aware.

6. Non-functional requirements

- Deterministic + reproducible; same inputs → same output.
- Always returns the **rate provenance** (rate_id, rate_as_of).
- Sub-millisecond per computation (cacheable rates).
- Missing-rate handling: return null amount + reason, never a wrong number.

7. Acceptance criteria

- **MUST** price every supported KPI/event where a valid rate exists.
- **MUST** select the effective-dated, most-specific rate and return its provenance (D7).
- **MUST** degrade safely (no rate → no $, flagged) — never fabricate.
- **MUST** be consumable by UIL, Reporting, and Monitoring alerts uniformly.
- **SHOULD** support multi-currency + per-asset overrides.

8. Build phasing

Phase 2: cost-rate schema + seeding tooling + computation service + integration into UIL/Reporting/alerts.

9. Risks

Stale/wrong rates mislead executives (mitigate: effective-dating, “rate as of” everywhere, client ownership + review cadence per D7); currency/timezone edge cases; double-counting (single computation service, no per-consumer math).

08 · Platform & Security (Cross-cutting) — Developer Specification

Component owner: Platform · **Phase:** 0 (foundational) · **Consumed by:** every component

1. Purpose & scope

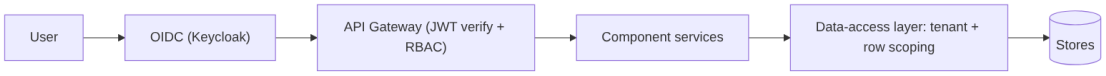
The cross-cutting foundation every service builds on: multi-tenancy, authentication/authorization, API gateway, deployment, observability, audit. **In scope:** these platform concerns + their contracts. **Out of scope:** component business logic.

2. Multi-tenancy (D8)

- **Logical isolation default**, physical on request. tenant_id (UUID) on **every** record + every event + every object-store prefix.
- Tenant scoping enforced at a **data-access layer** (row-level security in Postgres + mandatory tenant filter in repositories) — not left to callers.
- Per-tenant configuration object: deployment mode (D1), latency target (D2), retention, enabled modules, isolation level, branding, cost-rate ownership.
- No cross-tenant query path exists; cross-tenant access attempts are denied + audited.

3. Authentication & RBAC

- **OIDC** (Keycloak) — matches M1 SSO; supports on-prem (D1). Services validate JWT; short-lived tokens.
- **Roles** (extend M1): OPERATOR · SUPERVISOR · PLANT_HEAD · ADMIN + platform IMPLEMENTATION (connectors/mappings/templates) and CLIENT_ADMIN (uploads/exports/config).
- **Authorization model:** (role, resource, action) + **row-level line/site scoping** (reuse M1 line_access). Permissions checked at the gateway + service.



diagram

4. API gateway & conventions

REST /v1, OpenAPI per service, OIDC-secured, rate-limited, RFC-7807 errors, idempotency keys on writes, cursor pagination, correlation-id propagation. Internal service-to-service via mTLS + service identity.

5. Deployment (D1 hybrid)

- All services **containerized** (Docker), **12-factor**, externalized config + secrets (Vault).
- **Cloud:** Kubernetes. **On-prem/edge:** k3s or Compose for capture + PLC + a local cache, syncing to core.
- Per-client topology chosen from config; **no cloud-only assumptions** anywhere.
- CI/CD with environment promotion; DB migrations versioned (Flyway/Alembic).

6. Observability

OpenTelemetry traces, Prometheus metrics, structured logs (Loki/ELK) — all tagged tenant_id + component + correlation_id. Golden signals per service; pipeline + readiness dashboards (Grafana). Alerting on platform health.

7. Audit & lineage

Every write, mapping/override, export, and distribution is audit-logged (who, what, when, before/after) — generalizing the M1 audit pattern. Data lineage end-to-end (canonical record → batch → source row → mapping version).

8. Non-functional baseline (inherited by all)

Multi-tenant, hybrid-deployable, near-real-time (minutes) default latency, 99.5% serving availability v1, horizontal scale, TLS + at-rest encryption, least privilege, idempotent/replayable data flows, full observability.

9. Acceptance criteria

- **MUST** stamp + enforce tenant_id on every record and API call; no cross-tenant path.
- **MUST** authenticate via OIDC and authorize by role + row scope on every request.
- **MUST** run every service containerized and deployable cloud **or** on-prem (D1).
- **MUST** emit traces/metrics/logs tagged by tenant + component.
- **MUST** audit every write/export/mapping/distribution.
- **SHOULD** support physical tenant isolation on request (D8).

10. Build phasing

Phase 0: OIDC + gateway + RBAC + tenant data-access layer + config service + observability baseline + CI/CD + migrations. Everything else builds on this.

11. Risks

Tenant-isolation leaks (enforce centrally + test); on-prem drift from cloud (one container set, config-driven); secret sprawl (Vault); auth complexity on edge (cache tokens, offline-tolerant like M1).

09 · Glossary

Term	Meaning
Data Lake	The storage + processing + serving substrate; the platform's core wedge. Zones: Raw → Standardized → Canonical → Serving.
Manifold	The data-integration & mapping engine inside the Data Lake (connectors, mapping,

	classification, validation, dedup/MDM, conformance, readiness).
Canonical Data Model	The single shared schema every module speaks; standards-anchored (ISA-95, ISO 22400).
Event Spine	The universal canonical Event entity that all time-stamped facts specialize from (state, downtime, count, quality, energy, sensor…).
Unified Intelligence (UIL)	The synthesis layer: KPIs, cross-module correlation, trends, financial impact, insights.
Operational Monitoring	The live, low-latency "what's happening now" layer off the streaming path.
Reporting	Formatted, scheduled, distributable documents bound to the shared KPI registry.
Financial Impact	Cross-cutting service that prices KPIs/events using effective-dated cost-rates.
Zone (Raw/Standardized/Canonical/Serving)	The four Data-Lake tiers (medallion); each with a strict contract.
Golden record	The single resolved master record for an entity after dedup + survivorship.
Source-key map	Mapping of every (source_system, native_key) to one canonical surrogate id.
Readiness %	Per tenant×module score (Required gate + Additional + Quality + History) deciding deployability.
Data contract	A module's declared Required/Additional canonical entities/fields.
Sector template	A reusable mapping bundle per industry (steel seeded from M1) for fast onboarding.
Write-back	Modules pushing derived intelligence (e.g. risk score, downtime class) into the Canonical zone.
OEE	Overall Equipment Effectiveness = Availability × Performance × Quality (ISO 22400).
Six Big Losses	Loss taxonomy mapping reason codes to OEE components (Availability/Performance/Quality).

ISA-95	Equipment hierarchy: Enterprise → Site → Area → Work Center → Work Unit.
ISO 22400	Standard manufacturing KPI definitions.
COIL_NO	M1 traceability spine (steel); generalizes to canonical asset/material identity.
M1–M7 (platform)	Modules: M1 Shopfloor Digitization, M2 Maintenance, M3 Shopfloor Ops, M4 OEE/Downtime, M5 Yield, M6 Quality, M7 Energy.
M1 blueprint M2–M7	*Different numbering* — the Hero Steels downstream roadmap; do not confuse with platform M2–M7.
D1–D11	The locked architectural decisions (see 00 §3).
Zinance (FAMS)	The user's other product; its import/export module is reused UX prior art (D10).
CDC	Change Data Capture (incremental ERP sync).
MDM	Master Data Management (dedup + survivorship → golden record).
NFR	Non-Functional Requirement.